

FairSplit+ Database Schema in SQL

Beschrijving van het databaseschema van FairSplit+, inclusief tabellen, relaties en constraints. Het schema is opgebouwd in **SQL-DDL** en vormt de basis voor gebruikers, groepen, uitgaven en afrekeningen. Achter de bespreking van het schema wordt **PostgreSQL (Supabase)** die supabase zelf teruggeeft ook weergegeven.

- Row Level Security (RLS) is ingeschakeld voor alle tabellen.
- Er is een development policy actief die alle acties toestaat.
- Het schema is handmatig aangemaakt via SQL in Supabase.

Overzicht entiteiten

- **users** – Gebruikers van de applicatie
- **groups** – Groepen waarin kosten worden gedeeld
- **group_members** – Koppelt gebruikers aan groepen
- **expenses** – Uitgaven binnen een groep
- **expense_items** – Detailregels per uitgave (voor toekomstige AI-functionaliteit)
- **expense_shares** – Verdeling van kosten per gebruiker
- **payments** – Werkelijke betalingen tussen gebruikers

1. USERS

Bevat basisinformatie over gebruikers.

```
CREATE TABLE users (  
  user_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  phone_number VARCHAR(30),  
  payment_method VARCHAR(50)  
);
```

Opmerkingen:

- payment_method wordt gebruikt voor IBAN-nummer.
- Elke gebruiker heeft een uniek user_id.

2. GROUPS

Definieert een groep waarin kosten worden gedeeld.

```
CREATE TABLE groups (  
  group_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  start_date DATE NOT NULL,  
  end_date DATE NOT NULL,  
  organizer_id INTEGER NOT NULL REFERENCES users(user_id),  
  app_fee DECIMAL(10,2) DEFAULT 0.00,  
  icon TEXT  
);
```

Opmerkingen:

- Elke groep heeft exact één organisator.
 - app_fee krijgt momenteel via create_group een standaard waarde 3, maar kan bij verdere ontwikkeling een eventueel specifieke waarde krijgen in deze table, per groep of naargelang abonnement.
-

3. GROUP_MEMBERS

Koppelt gebruikers aan groepen en bepaalt hun rol.

```
CREATE TABLE group_members (  
  group_id INTEGER NOT NULL REFERENCES groups(group_id) ON DELETE CASCADE,  
  user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
  role VARCHAR(20) DEFAULT 'member', -- 'organizer' of 'member'  
  joined_date DATE DEFAULT CURRENT_DATE,  
  PRIMARY KEY (group_id, user_id)  
);
```

Opmerkingen:

- Samengestelde primaire sleutel voorkomt dubbele leden.

- Cascade delete zorgt voor automatische opschoning bij verwijderen van een groep of gebruiker.

4. EXPENSES

Registreert uitgaven binnen een groep.

```
CREATE TABLE expenses (  
  expense_id SERIAL PRIMARY KEY,  
  group_id INTEGER NOT NULL REFERENCES groups(group_id) ON DELETE CASCADE,  
  paid_by INTEGER NOT NULL REFERENCES users(user_id),  
  description TEXT,  
  total_amount DECIMAL(10,2) NOT NULL,  
  receipt_image TEXT  
);
```

Opmerkingen:

- paid_by is de persoon die het bedrag heeft voorgeschooten.
- Kostenverdeling gebeurt via expense_shares.
- receipt_image kan gebruikt worden voor toekomstige functie waarbij bonnetjes ingescanned kunnen worden en.

5. EXPENSE_ITEMS

Net zoals receipt_image is deze table voor het extraheren van line items uit bonnetjes (Momenteel niet nog geen functie in de code)

```
CREATE TABLE expense_items (  
  item_id SERIAL PRIMARY KEY,  
  expense_id INTEGER NOT NULL REFERENCES expenses(expense_id) ON DELETE CASCADE,  
  description TEXT NOT NULL,  
  quantity INTEGER DEFAULT 1,  
  unit_price DECIMAL(10,2) NOT NULL,  
  total DECIMAL(10,2) GENERATED ALWAYS AS (quantity * unit_price) STORED  
);
```

Opmerkingen:

- Voor het opsplitsen van uitgaven in afzonderlijke items.
- ON DELETE CASCADE zorgt dat items verdwijnen als de bijbehorende expense wordt verwijderd.

6. EXPENSE_SHARES

Geeft aan welk deel van een uitgave iedere gebruiker verschuldigd is.

```
CREATE TABLE expense_shares (  
  expense_id INTEGER NOT NULL REFERENCES expenses(expense_id) ON DELETE CASCADE,  
  user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
  amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),  
  PRIMARY KEY (expense_id, user_id)  
);
```

Opmerkingen:

- Zorgt ervoor dat kosten nooit negatief kunnen zijn.
- Essentieel voor saldo- en afrekenlogica.
- Omdat expense_items nog niet actief is, wordt item_id nog niet gebruikt in deze table.

7. PAYMENTS

Registreert daadwerkelijke betalingen tussen gebruikers.

```
CREATE TABLE payments (  
  payment_id SERIAL PRIMARY KEY,  
  from_user INTEGER NOT NULL REFERENCES users(user_id),  
  to_user INTEGER NOT NULL REFERENCES users(user_id),  
  group_id INTEGER REFERENCES groups(group_id),  
  amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),  
  method VARCHAR(30), -- bv. 'Payconiq', 'Revolut', 'IBAN'  
  status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'completed',  
'failed')),  
  payment_date DATE DEFAULT CURRENT_DATE,  
  reference TEXT  
);
```

Opmerkingen:

- Wordt gebruikt voor echte afrekeningen.
- method en reference zijn voorbereid voor eventuele uitbreidingen, maar standaard gebruiken binnen de MVP bankbetalingen.
- Betalingen beïnvloeden de uiteindelijke saldi, maar vervangen geen expenses.

PostgreSQL(Supabase) SCHEMA

-- =====

-- USERS

-- =====

```
create table public.users (  
    user_id serial not null,  
    name character varying(100) not null,  
    phone_number character varying(30) null,  
    payment_method character varying(50) null,  
    constraint users_pkey primary key (user_id)  
) TABLESPACE pg_default;
```

-- =====

-- GROUPS

-- =====

```
create table public.groups (  
    group_id serial not null,  
    name character varying(100) not null,  
    start_date date not null,  
    end_date date not null,  
    organizer_id integer not null,  
    app_fee numeric(10, 2) null default 0.00,  
    icon text null,  
  
    constraint groups_pkey primary key (group_id),  
    constraint groups_organizer_id_fkey foreign KEY (organizer_id) references users  
(user_id)  
) TABLESPACE pg_default;
```

-- =====

-- GROUP MEMBERS

-- =====

```
create table public.group_members (  
    group_id integer not null,  
    user_id integer not null,  
    role character varying(20) null default 'member'::character varying,  
    joined_date date null default CURRENT_DATE,
```

```

        constraint group_members_pkey primary key (group_id, user_id),
        constraint group_members_group_id_fkey foreign KEY (group_id) references groups
(group_id) on delete CASCADE,
        constraint group_members_user_id_fkey foreign KEY (user_id) references users
(user_id) on delete CASCADE
    ) TABLESPACE pg_default;

-- =====

-- EXPENSES

-- =====

create table public.expenses (
    expense_id serial not null,
    group_id integer not null,
    paid_by integer not null,
    description text null,
    total_amount numeric(10, 2) not null,
    receipt_image text null,

    constraint expenses_pkey primary key (expense_id),
    constraint expenses_group_id_fkey foreign KEY (group_id) references groups
(group_id) on delete CASCADE,
    constraint expenses_paid_by_fkey foreign KEY (paid_by) references users (user_id)
) TABLESPACE pg_default;

-- =====

-- EXPENSE ITEMS

-- =====

create table public.expense_items (
    item_id serial not null,
    expense_id integer not null,
    description text not null,
    quantity integer null default 1,
    unit_price numeric(10, 2) not null,
    total numeric generated always as (quantity * unit_price) stored,

    constraint expense_items_pkey primary key (item_id),
    constraint expense_items_expense_id_fkey foreign key (expense_id) references
expenses (expense_id) on delete cascade
) TABLESPACE pg_default;

-- =====

-- EXPENSE SHARES

-- =====

create table public.expense_shares (

```

```

expense_id integer not null,
user_id integer not null,
amount numeric(10, 2) not null,

constraint expense_shares_pkey primary key (expense_id, user_id),
constraint expense_shares_expense_id_fkey foreign key (expense_id) references
expenses (expense_id) on delete cascade,
constraint expense_shares_user_id_fkey foreign key (user_id) references users
(user_id) on delete cascade,
constraint expense_shares_amount_check check (amount >= 0)
);

-- =====

-- PAYMENTS

-- =====

create table public.payments (
    payment_id serial not null,
    from_user integer not null,
    to_user integer not null,
    group_id integer null,
    amount numeric(10, 2) not null,
    method character varying(30) null,
    status character varying(20) null default 'pending',
    payment_date date null default CURRENT_DATE,
    reference text null,

    constraint payments_pkey primary key (payment_id),
    constraint payments_from_user_fkey foreign key (from_user) references users
(user_id),
    constraint payments_to_user_fkey foreign key (to_user) references users (user_id),
    constraint payments_group_id_fkey foreign key (group_id) references groups
(group_id),
    constraint payments_amount_check check (amount > 0),
    constraint payments_status_check check (
        status in ('pending', 'completed', 'failed')
    )
);

```